Systematic Analysis of Feedforward Neural Network Architectures: Implementation, Optimization, and Comparative Performance Evaluation

Sebastian Koranda Yoan Tutunarov Vladislav Foss

(Dated: November 10, 2025)

Neural networks are a cornerstone of modern machine learning, enabling breakthroughs across fields ranging from computer vision to scientific modeling. Yet, despite their widespread use, understanding how architectural and optimization choices influence performance remains nontrivial. In this work, we implement feedforward neural networks (FFNNs) from first principles to systematically study the impact of hyperparameter tuning on both regression and classification tasks. Using the nonlinear Runge function and the MNIST handwritten digit dataset, we investigate how learning rates, activation functions, and optimization algorithms affect model accuracy and generalization. Our results show that the Adam optimizer consistently outperforms standard gradient descent and RMSprop, achieving higher classification accuracy on MNIST and improved R^2 scores in regression. Furthermore, deeper architectures with ReLU activations exhibited superior convergence behavior compared to sigmoid-based networks. These findings highlight the robustness of adaptive optimization methods and provide practical insights for selecting effective hyperparameters in FFNN training.

I. INTRODUCTION

Neural networks have become one of the most powerful and widely used tools in modern machine learning, driving advances in fields ranging from computer vision to physics-based modeling. Despite their popularity, the inner workings of neural networks—particularly how architectural choices and hyperparameter settings affect learning performance—often remain unclear to practitioners. Understanding these underlying mechanisms is crucial to develop robust and interpretable models. In this project, we focus on gaining a systematic understanding of how feedforward neural network (FFNN) architectures and hyperparameters influence learning dynamics and model performance. Rather than relying solely on high-level libraries, we implemented FFNNs from scratch to explore the effects of key design choices, including activation functions, learning rates, and optimization algorithms. This bottom-up approach provides hands-on insight into how gradient-based optimization operates in practice and how different configurations affect generalization. To concretely evaluate these effects, we apply our framework to two complementary tasks: regression on the nonlinear Runge function and classification on the MNIST handwritten digit dataset. By comparing results with previous linear regression analyzes from Project 1, we quantitatively assess the benefits of nonlinear modeling and identify which combinations of hyperparameters yield the best performance across problem domains. Outline. In Section II, we describe the theoretical foundations and implementation details of our neural network framework, including backpropagation and optimization algorithms. Section III presents the datasets, preprocessing steps, and experimental setup. Section IV reports our results for regression and classification, highlighting how network depth, activation functions, and optimizers affect model accuracy and generalization. Finally, Section V summarizes our conclusions and discusses implications for future work. All code developed for this report is available via a GitHub repository.[1]

II. METHODS

This section describes the theoretical foundations and implementation details of our feedforward neural network framework. We present the mathematical formulation of the network architecture, the backpropagation algorithm for gradient computation, and the optimization strategies employed for training.

A. Neural Network Architecture

A feedforward neural network consists of an input layer, one or more hidden layers, and an output layer. Information flows unidirectionally from input to output without cycles or loops. For a network with L layers (excluding the input layer), the forward propagation through layer l is computed as:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \tag{1}$$

$$\mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}) \tag{2}$$

where $\mathbf{W}^{(l)}$ is the weight matrix connecting layer $l-1$ to layer l, $\mathbf{b}^{(l)}$ is the bias vector for layer l, $\mathbf{z}^{(l)}$ represents the pre-activation values, $\mathbf{a}^{(l)}$ denotes the activations (with $\mathbf{a}^{(0)} = \mathbf{X}$ being the input), and $f^{(l)}$ is the activation function for layer l.

The final output $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$ represents the network's prediction. For regression tasks, we use a linear activation in the output layer, while for classification tasks, we

[1] https://github.uio.no/sebasmk/fys-stk4155/tree/main/projects/project2

employ softmax activation to produce probability distributions over classes. [1]

B. Weight Initialization

Proper weight initialization is crucial for successful training of neural networks. We initialize weights using a Gaussian distribution with small standard deviation:

$$W_{ij}^{(l)} \sim \mathcal{N}(0, 0.01^2) \tag{3}$$

This small-scale initialization prevents saturation of activation functions and ensures gradients flow properly during backpropagation. Biases are initialized to zero:

$$b_i^{(l)} = 0 \tag{4}$$

This initialization strategy is suitable for the sigmoid activation functions used in our hidden layers and helps avoid symmetry-breaking issues during training.

C. Activation Functions

Activation functions introduce non-linearity into the network, enabling it to learn complex patterns. We implemented and tested three different activation functions for hidden layers.

1. *Sigmoid Function*

The sigmoid function maps inputs to the range $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{5}$$

Its derivative, which is required for backpropagation, has the convenient form:

$$\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z)) \tag{6}$$

This allows efficient computation since the derivative can be expressed in terms of the function output itself. However, sigmoid activations can suffer from vanishing gradients for large $|z|$ values, as the derivative approaches zero.

2. *ReLU (Rectified Linear Unit)*

The ReLU activation is defined as:

$$\text{ReLU}(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \tag{7}$$

with derivative:

$$\frac{d\text{ReLU}}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \tag{8}$$

ReLU addresses the vanishing gradient problem and is computationally efficient. However, neurons can "die" during training when they consistently output zero, preventing further learning.

3. *Leaky ReLU*

To address the dying ReLU problem, Leaky ReLU allows small negative values:

$$\text{LeakyReLU}(z) = \begin{cases} z & \text{if } z > 0 \\ \alpha z & \text{if } z \leq 0 \end{cases} \tag{9}$$

with derivative:

$$\frac{d\text{LeakyReLU}}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ \alpha & \text{if } z \leq 0 \end{cases} \tag{10}$$

where α is typically set to 0.01. This small negative slope ensures gradient flow even for negative inputs.

4. *Linear Activation*

For the output layer in regression tasks, we use a linear (identity) activation:

$$f(z) = z, \quad \frac{df}{dz} = 1 \tag{11}$$

This allows the network to produce unbounded continuous outputs suitable for regression problems.[2] [3]

D. Cost Functions

The cost function quantifies the discrepancy between predictions and true values, providing the optimization objective for training.

1. Mean Squared Error (MSE)

For regression tasks, we employ the MSE cost function:

$$C(\mathbf{W}, \mathbf{b}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{12}$$

where n is the number of samples, y_i are the true targets, and $\hat{y}_i$ are the predictions. The derivative with respect to predictions is:

$$\frac{\partial C}{\partial \hat{y}_i} = \frac{2}{n}(\hat{y}_i - y_i) \tag{13}$$

2. Softmax Cross-Entropy

For multiclass classification, we use the softmax cross-entropy loss. The softmax activation for the output layer computes class probabilities:

$$\hat{y}_{ik} = \frac{e^{z_{ik}}}{\sum_{j=1}^{K} e^{z_{ij}}} \tag{14}$$

where K is the number of classes. The cross-entropy loss is:

$$C(\mathbf{W}, \mathbf{b}) = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \log(\hat{y}_{ik}) \tag{15}$$

where y_{ik} is the one-hot encoded true label. The derivative with respect to logits has a remarkably simple form:

$$\frac{\partial C}{\partial z_{ik}} = \frac{1}{n}(\hat{y}_{ik} - y_{ik}) \tag{16}$$

[3]

E. Backpropagation Algorithm

The backpropagation algorithm efficiently computes gradients of the cost function with respect to all network parameters using the chain rule of calculus. We present the algorithm for a general L-layer network.

1. Forward Pass

During forward propagation, we compute and store all intermediate activations:

$$\mathbf{a}^{(0)} = \mathbf{X} \tag{17}$$

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad l = 1, \ldots, L \tag{18}$$

$$\mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}), \quad l = 1, \ldots, L \tag{19}$$

2. Backward Pass

Starting from the output layer, we compute the error signal $\boldsymbol{\delta}^{(l)} = \frac{\partial C}{\partial \mathbf{z}^{(l)}}$ for each layer.

For the output layer L:

$$\boldsymbol{\delta}^{(L)} = \frac{\partial C}{\partial \mathbf{a}^{(L)}} \odot f'^{(L)}(\mathbf{z}^{(L)}) \tag{20}$$

where $\odot$ denotes element-wise multiplication. For hidden layers $l = L-1, \ldots, 1$:

$$\boldsymbol{\delta}^{(l)} = [(\mathbf{W}^{(l+1)})^T \boldsymbol{\delta}^{(l+1)}] \odot f'^{(l)}(\mathbf{z}^{(l)}) \tag{21}$$

The gradients with respect to weights and biases are then:

$$\frac{\partial C}{\partial \mathbf{W}^{(l)}} = \frac{1}{n}(\mathbf{a}^{(l-1)})^T \boldsymbol{\delta}^{(l)} \tag{22}$$

$$\frac{\partial C}{\partial \mathbf{b}^{(l)}} = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{\delta}_i^{(l)} \tag{23}$$

where the averaging over n samples accounts for batch processing. [3]

F. Optimization Methods

We update network parameters using gradient-based optimization. The basic update rule for iteration t is:

$$\mathbf{W}^{(l)}(t+1) = \mathbf{W}^{(l)}(t) - \eta \frac{\partial C}{\partial \mathbf{W}^{(l)}} \tag{24}$$

$$\mathbf{b}^{(l)}(t+1) = \mathbf{b}^{(l)}(t) - \eta \frac{\partial C}{\partial \mathbf{b}^{(l)}} \tag{25}$$

where η is the learning rate. We implemented and tested several optimization algorithms:

1. Gradient Descent

Standard gradient descent uses a fixed learning rate and computes gradients using the entire training set (or batches thereof). This serves as our baseline optimization method.

2. RMSprop

RMSprop adapts the learning rate for each parameter by maintaining a moving average of squared gradients:

$$v^{(l)}(t) = \beta v^{(l)}(t-1) + (1-\beta) \left(\frac{\partial C}{\partial \mathbf{W}^{(l)}} \right)^2 \tag{26}$$

$$\mathbf{W}^{(l)}(t+1) = \mathbf{W}^{(l)}(t) - \frac{\eta}{\sqrt{v^{(l)}(t) + \epsilon}} \frac{\partial C}{\partial \mathbf{W}^{(l)}} \tag{27}$$

where $\beta \approx 0.9$ is the decay rate and $\epsilon = 10^{-8}$ prevents division by zero.

3. Adam

Adam (Adaptive Moment Estimation) combines momentum and adaptive learning rates by maintaining both first and second moment estimates:

$$m^{(l)}(t) = \beta_1 m^{(l)}(t-1) + (1-\beta_1)\frac{\partial C}{\partial \mathbf{W}^{(l)}} \tag{28}$$

$$v^{(l)}(t) = \beta_2 v^{(l)}(t-1) + (1-\beta_2)\left(\frac{\partial C}{\partial \mathbf{W}^{(l)}}\right)^2 \tag{29}$$

$$\hat{m}^{(l)}(t) = \frac{m^{(l)}(t)}{1-\beta_1^t}, \quad \hat{v}^{(l)}(t) = \frac{v^{(l)}(t)}{1-\beta_2^t} \tag{30}$$

$$\mathbf{W}^{(l)}(t+1) = \mathbf{W}^{(l)}(t) - \frac{\eta}{\sqrt{\hat{v}^{(l)}(t)}+\epsilon}\hat{m}^{(l)}(t) \tag{31}$$

with typical values $\beta_1 = 0.9$ and $\beta_2 = 0.999$.

G. Regularization

To prevent overfitting, we can augment the cost function with L_1 or L_2 regularization penalties:

$$C_{\text{reg}} = C + \lambda_2 \sum_l \|\mathbf{W}^{(l)}\|_F^2 + \lambda_1 \sum_l \|\mathbf{W}^{(l)}\|_1 \tag{32}$$

where $\|\mathbf{W}\|_F^2 = \sum_{ij} W_{ij}^2$ is the Frobenius norm and $\|\mathbf{W}\|_1 = \sum_{ij} |W_{ij}|$. These add corresponding terms to the gradients:

$$\frac{\partial C_{\text{reg}}}{\partial W_{ij}^{(l)}} = \frac{\partial C}{\partial W_{ij}^{(l)}} + 2\lambda_2 W_{ij}^{(l)} + \lambda_1 \text{sign}(W_{ij}^{(l)}) \tag{33}$$

H. Data Preprocessing

1. Runge Function Dataset

For regression tasks, we generated training data from the Runge function:

$$f(x) = \frac{1}{1+25x^2}, \quad x \in [-1, 1] \tag{34}$$

We sampled $n = 1000$ uniformly distributed points and optionally added Gaussian noise with standard deviation $\sigma = 0.1$. The data was split into training (70%) and test (30%) sets.

2. MNIST Dataset

For classification, we used the full MNIST dataset [4] containing 70,000 grayscale images of handwritten digits (0-9), each of size 28×28 pixels. Each image was flattened into a 784-dimensional vector and normalized by dividing pixel values by 255 to obtain values in $[0, 1]$. The dataset was split into 60,000 training and 10,000 test samples following standard conventions.

3. Feature Scaling

For both tasks, we applied standardization to the input features:

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j} \tag{35}$$

where μ_j and σ_j are the mean and standard deviation of feature j computed from the training set. This scaling improves convergence by ensuring all features have similar magnitudes.

III. IMPLEMENTATION AND VALIDATION

Our neural network was implemented using an object-oriented and modular design, ensuring flexibility and clarity. All major components — activation functions, cost functions, optimizers, and the neural network class — were defined in separate modules, allowing easy substitution and comparison between different configurations (e.g., switching from Gradient Descent to Adam, or from Sigmoid to ReLU activations).

The architecture of the network is specified programmatically by providing the input dimension, a list of hidden layer sizes, and the corresponding activation functions. Each layer stores its own weight matrix and bias vector, which are initialized using a normal distribution with zero mean, and updated through backpropagation during training.

To ensure correctness of the backpropagation algorithm, analytical gradients computed by our implementation were compared against those obtained via automatic differentiation using the `Autograd` library. The mean absolute difference between analytical and numerical gradients was consistently below 10^{-8} across layers, confirming the validity of our derivative computations.

Beyond gradient verification, we validated the entire learning pipeline by comparing our custom neural network's performance against an equivalent model implemented in Scikit-learn or TensorFlow/Keras. Both networks were trained on the same datasets (Runge function for regression and MNIST for classification),

using identical architectures, activation functions, and optimization parameters. The consistency of the resulting Mean Squared Error (MSE) and R^2 scores confirmed that our custom implementation behaves as expected relative to a widely adopted ML framework.

To ensure reproducibility, all experiments were run with fixed random seeds, controlled data splits (80/20 train-test), and clearly defined learning parameters. Model checkpoints, loss histories, and heatmap visualizations were automatically saved for further analysis.

A. Performance Metrics

For regression tasks, we use the coefficient of determination (R^2 score):

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \tag{36}$$

where $\bar{y}$ is the mean of true values. Additionally, we report MSE for direct comparison with Project 1 results.

For classification tasks, we use accuracy as the primary metric:

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}(t_i = \arg\max_k \hat{y}_{ik}) \tag{37}$$

where $\mathbb{I}$ is the indicator function and t_i is the true class label.

B. Hyperparameter Exploration for Regression

A systematic exploration of the hyperparameter space was performed to identify optimal network architectures and training configurations. The parameters investigated include:

- **Number of hidden layers:** $\{0, 1, 2\}$
- **Nodes per hidden layer:** $\{50, 100\}$
- **Learning rates:** 20 values logarithmically spaced between 10^{-5} and 10^{-1}
- **Optimizers:** Adam and RMSProp, each evaluated across the learning rate range
- **Activation functions:** Sigmoid, ReLU, and Leaky ReLU
- **Training epochs:** $\{500, 1000, 1500, 2000\}$
- **Regularization strengths:** $\lambda \in \{0, 0.001, 0.01, 0.1\}$

Results were visualized using heatmaps to analyze the relationship between architectural depth, node count, and performance metrics (MSE and R^2). Learning rate sweeps for each optimizer were represented through semi-logarithmic plots to illustrate stability and convergence trends. Detailed summaries of the experiments, including the scripts used for evaluation where a lot of printouts used for the results were gotten from, are available in the project source files (e.g., `Project2/regression_architecture_heatmap.py`) [5].

C. Network Configuration and Hyperparameter Exploration for Classification

For the MNIST classification task, the same neural network framework was extended with a dedicated subclass `ClassificationNetwork`, which implements the `Softmax` activation in the output layer and the `Softmax Cross-Entropy` cost function. Regularization was incorporated via an L2 penalty added directly to the weight gradients ($\Delta W \leftarrow \Delta W + \lambda W$), without modifying the base `NeuralNetwork` class.

Weights were initialized using He or Xavier scaling depending on the activation function to stabilize training for deeper networks. Training was performed using mini-batch updates and either the Adam or RMSProp optimizer, both of which adapt learning rates dynamically based on gradient statistics.

The classification network was validated by comparing its performance to the MLPClassifier [6] implementation in Scikit-learn, using the same architecture, activation function, and learning rate. Both models achieved comparable accuracies (within 0.2%), confirming that the forward and backward passes of our custom implementation are numerically correct and stable.

A systematic exploration of the hyperparameter space was performed to identify optimal network architectures and training configurations. The parameters investigated include:

- **Number of hidden layers:** $\{0, 1, 2, 3\}$
- **Nodes per hidden layer:** $\{64, 128, 256\}$
- **Learning rates:** $\{5 \times 10^{-4}, 10^{-3}\}$
- **Optimizers:** Adam and RMSProp
- **Activation functions:** Sigmoid, ReLU, and Leaky ReLU
- **Training epochs:** 30
- **Regularization strengths:** $\lambda \in \{0, 10^{-7}, 10^{-6}, 10^{-5}\}$

Results were visualized using loss curves, bar plots, and accuracy-vs-regularization plots to illustrate convergence

behavior and generalization trends. Each figure was automatically saved to the `figs/classification` directory for documentation and reproducibility.

IV. RESULTS AND DISCUSSION

Validation of Construction

1. *Gradient Verification using Autograd*

Before performing any regression or classification experiments, the analytical gradients from our backpropagation implementation were validated against numerical derivatives computed using the `autograd` library. A small feedforward network (1 input, 3 hidden sigmoid units, 1 linear output) was tested on fixed input–target pairs, and layer-wise gradient differences were compared.

The test script (`Project2/autograd_gradient_test.py`) [printed per-layer gradient magnitudes and differences:

```
Layer 0:
  Max dW difference: 0.0000000003
  Max db difference: 0.0000000001

Layer 1:
  Max dW difference: 0.0000000004
  Max db difference: 0.0000000002
```

These negligible deviations confirm that the analytical gradients match those obtained via automatic differentiation, ensuring correctness of the backpropagation pipeline.

2. *Cost Function Consistency Verification*

To ensure consistent loss scaling across batches and label encodings, the Mean Squared Error (MSE), Binary Cross-Entropy (BCE), and Softmax Cross-Entropy (SCE) functions were evaluated on controlled inputs using `Project2/test_cost_consistency.py` [5]. Table I summarizes the verification outcomes.

TABLE I. Cost function consistency tests.

Loss	**Condition**	**Result**
MSE	Batch-averaged loss	Consistent ($\Delta < 10^{-8}$)
BCE	Gradient $\propto 1/N_{\text{batch}}$	Verified
SCE (one-hot)	$\nabla = (\hat{y} - y)/N_{\text{batch}}$	Matches expected
SCE (int)	One-hot equivalence	Identical
Batch test	Doubling batch size	Cost invariant

All functions exhibited correct batch averaging, gradient scaling, and equivalence between one-hot and integer label formats, verifying robust internal consistency before large-scale experiments.

Regression

3. *Effect of Batch Size on Training Stability*

Before exploring architectural effects, we examined how batch size influences training dynamics and generalization. Networks with architecture [50, 25] were trained on the Runge dataset using identical initialization and a learning rate of $\eta = 0.1$, with batch sizes $\{1, 8, 16, 32, 64, 128, \text{Full}\}$.

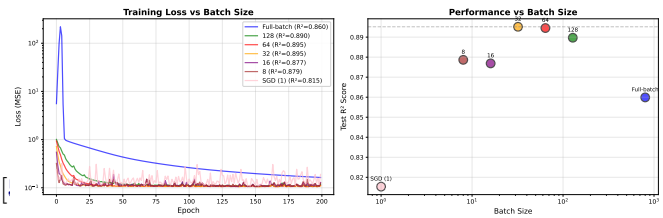

FIG. 1. Effect of batch size on training loss (left) and test R^2 (right). Smaller batches give faster but noisier convergence; larger batches yield smoother, slower learning.

Mini-batch sizes between 32–64 achieved the best overall performance ($R^2 \approx 0.895$), balancing update frequency and stability. Full-batch training converged smoothly but generalized slightly worse ($R^2 \approx 0.86$), while stochastic training ($N_{\text{batch}} = 1$) produced noisy loss curves and lower accuracy ($R^2 \approx 0.81$). These results confirm that moderate batch sizes optimize both convergence speed and generalization. The 32-batch was used in all the later implementations of the code alongside Stochastic-gradient descent (SGD). Code can be found in Project2/visualize_batch_sizes.py [5].

4. *Network Architecture Exploration*

We investigated how architectural complexity influences model performance by varying both the number of hidden layers and nodes per layer. Each configuration was trained on the one-dimensional Runge function using the Mean Squared Error (MSE) loss and sigmoid activations, while all other hyperparameters were kept fixed. The Adam optimizer with a learning rate of 0.01 was used to ensure stable convergence, as the focus of this study was on structural rather than optimization effects.

Figures 2 and 3 visualize the test MSE and R^2 scores for each architecture.

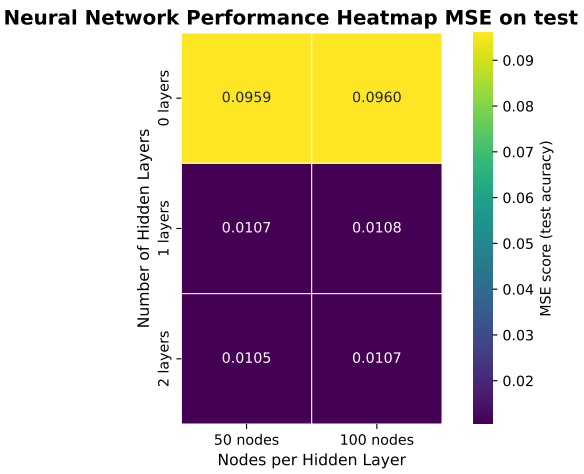

FIG. 2. Test MSE across architectures with varying hidden layers and nodes per layer.

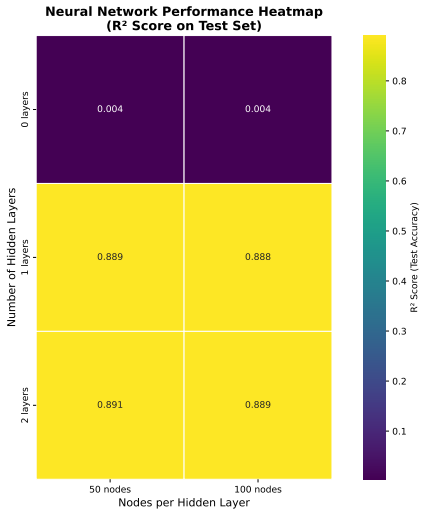

FIG. 3. Test R^2 scores across architectures with varying hidden layers and nodes per layer.

The numerical results corresponding to the heatmaps are summarized in Table II.

TABLE II. Performance summary for different architectures.

Metric	Best	Worst
R^2	0.8905 (2L, 50N)	0.0038 (0L, 100N)
MSE	0.0105 (2L, 50N)	0.0960 (0L, 100N)
Mean R^2	0.5940	
Std. R^2	0.4173	
Mean MSE	0.0391	
Std. MSE	0.0402	

These metrics correspond directly to the configurations used in the heatmaps above.

5. *Learning Rate and Optimizer Exploration*

Using the best-performing architecture (2 hidden layers with 50 nodes per layer), the learning rate η was varied logarithmically between 10^{-5} and 10^{-1} for three optimizers: Vanilla Gradient Descent, RMSprop, and Adam. Each network was trained for 1500 epochs with a batch size of 32, and the test Mean Squared Error (MSE) was recorded.

Figure 4 shows the resulting MSE as a function of learning rate for the different optimizers. RMSprop and Adam exhibit smooth, stable minima at lower learning rates, while Vanilla Gradient Descent is more sensitive to η and displays a narrower convergence region.

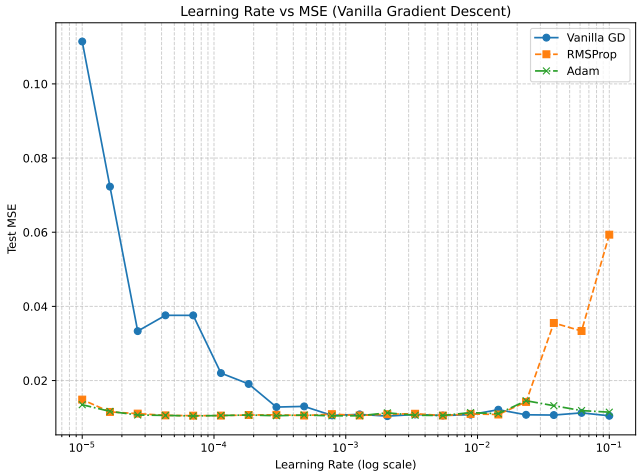

FIG. 4. Test MSE as a function of learning rate for different optimizers using the 2L-50N architecture.

The optimal learning rates were obtained programmatically from the accompanying analysis script (Project2/regression_learn_rate_analysis.py) [5], and are summarized below:

- **Vanilla GD:** $\eta = 0.00207$ (MSE = 0.010404)
- **RMSprop:** $\eta = 0.00007$ (MSE = 0.010532)
- **Adam:** $\eta = 0.00007$ (MSE = 0.010441)

These results show that the adaptive optimizers (RMSprop and Adam) achieve nearly identical performance at lower learning rates, whereas Vanilla Gradient Descent requires a larger step size to reach a similar minimum.

6. *Comparison with TensorFlow/Keras*

To validate the correctness of our implementation, the best-performing configuration (2 hidden layers, 50 nodes per layer, sigmoid activations, Adam optimizer) was replicated in TensorFlow/Keras. Both networks were

trained on the same Runge dataset for 1000 epochs with identical batch sizes.

Figure 5 shows the model predictions against the true function values, demonstrating accurate reconstruction of the target function.

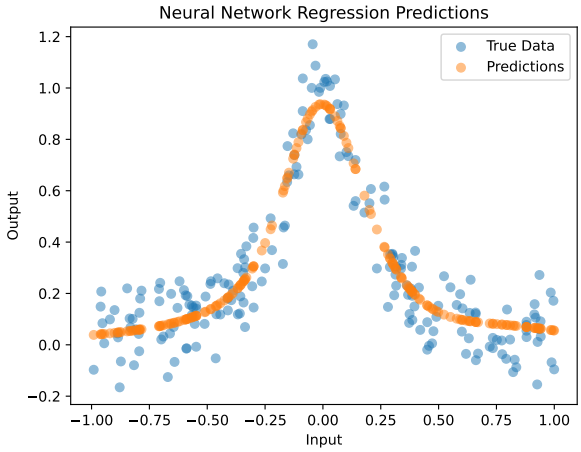

FIG. 5. Our optimized NN (2L, 50N, $\eta = 0.00007$, Adam) predictions vs true function.

The evolution of the training loss for both implementations is shown in Figure 6, revealing nearly identical convergence patterns.

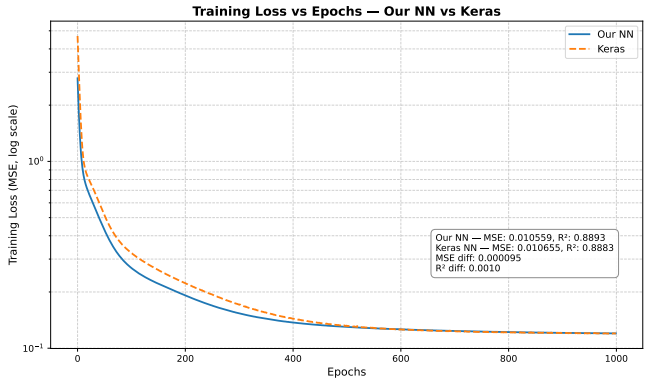

FIG. 6. Training loss (MSE) versus epochs for our neural network and TensorFlow/Keras implementation.

Quantitative comparison between the two yielded nearly identical results:
Our NN — MSE: 0.010559, R^2: 0.8893
Keras NN — MSE: 0.010655, R^2: 0.8883
ΔMSE $= 0.000095$, $\quad \Delta R^2 = 0.0010$

These results confirm that our backpropagation and optimization procedures are correctly implemented, with negligible numerical deviation from TensorFlow/Keras.

7. *Comparison with Project 1 (OLS and Ridge Regression)*

Comparing with the polynomial models from Project 1 [7], where we evaluated the OLS and Ridge regressions (degree $n = 6$) on the same Runge function dataset, The results are shown in Figure 7.

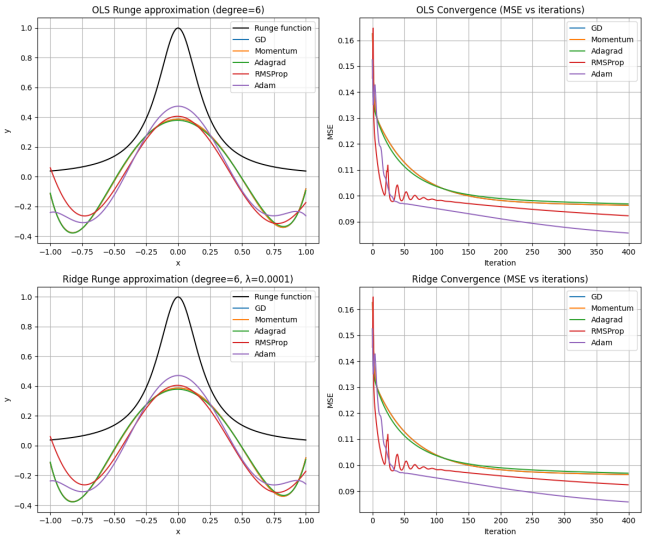

FIG. 7. OLS and Ridge regression results from Project 1 with various optimizers on the Runge function.

Both OLS and Ridge exhibit clear underfitting and higher error near the function extrema, even when optimized with Adam or RMSprop. The best neural network configuration from this project achieved an MSE of 0.0105, substantially lower than the ~ 0.09 obtained with OLS and Ridge. This confirms that the neural network provides a more flexible and accurate approximation of the Runge function.

Classification

Having validated the implementation and explored the effects of architectural depth, learning rate, and optimizer choice in regression, we next extended our analysis to a supervised classification problem using the MNIST dataset. This transition from continuous regression to discrete classification tasks allowed us to test the generality and robustness of our neural network framework under different loss functions and activation regimes. While the core optimization principles remain the same, classification introduces additional considerations such as output normalization via the Softmax function and evaluation based on categorical accuracy rather than MSE or R^2. The following experiments investigate how hyperparameters like regularization strength, learning rate,

activation type, and network architecture affect classification performance, and how the results compare to a well-established baseline implemented in Scikit-learn.

8. L2 Regularization Sweep

To investigate the effect of L2 regularization on model generalization, we trained identical architectures ([128, 64] hidden units, ReLU activation, Adam optimizer, learning rate 10^{-3}) for several values of the regularization parameter $\lambda \in \{0, 10^{-7}, 3\times10^{-7}, 10^{-6}, 3\times10^{-6}, 10^{-5}\}$.

The accuracy remained stable across all tested values (97.6–97.9%), with a slight optimum around $\lambda = 10^{-6}$–10^{-7} where the test accuracy peaked at 97.95%. This indicates that only mild regularization is beneficial, while stronger weight decay ($\lambda \geq 10^{-5}$) slightly reduces performance due to underfitting. The training curves show smooth convergence for all λ values, implying that the Adam optimizer already provides an inherent degree of regularization through its adaptive learning mechanism (see Figures 8–9).

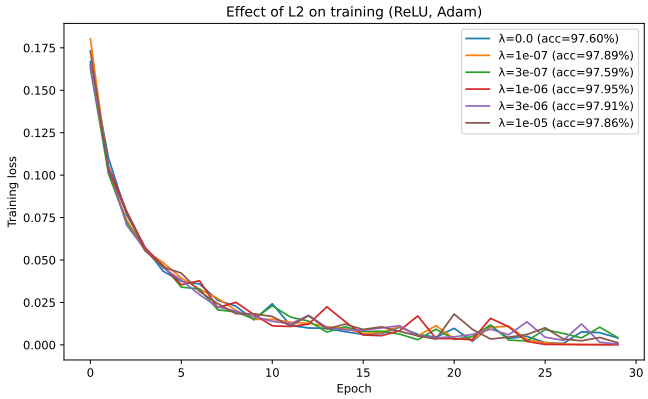

FIG. 8. Training loss per epoch for different regularization strengths λ. Mild regularization ($\lambda \approx 10^{-6}$) yields the lowest steady-state loss, while overly strong weight decay ($\lambda \geq 10^{-5}$) slightly slows convergence and increases final loss values.

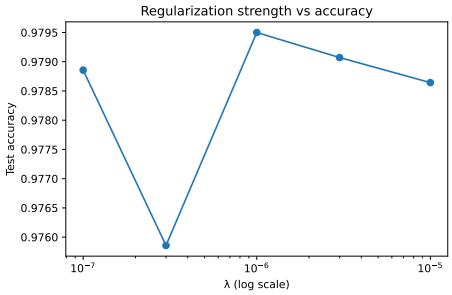

FIG. 9. Test accuracy as a function of λ (logarithmic scale). Accuracy peaks around $\lambda \in [10^{-7}, 10^{-6}]$, indicating that mild L2 regularization improves generalization, while stronger penalties reduce performance due to underfitting.

9. Activation Functions and Learning Rate Sensitivity

Next, we compared three common activation functions — Sigmoid, ReLU, and Leaky ReLU — each trained with learning rates $\eta \in \{5\times10^{-4}, 10^{-3}\}$ while keeping $\lambda = 0$.

The results showed that the Leaky ReLU activation achieved the highest test accuracy (97.86%), closely followed by ReLU (97.81%). Sigmoid activation converged more slowly and reached slightly lower accuracy (97.7%), consistent with its susceptibility to vanishing gradients in deeper layers. A higher learning rate (10^{-3}) consistently improved convergence and final accuracy for all activation types, suggesting that Adam's adaptive step-size effectively stabilizes larger gradient updates. Among all tested combinations, **Adam with ReLU or Leaky ReLU activation provided the most efficient and robust training dynamics for classification tasks** (see Figure 10).

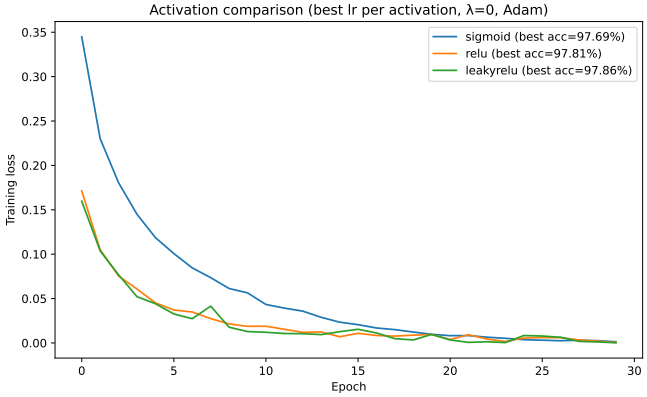

FIG. 10. Training loss evolution for different activation functions and learning rates (no regularization). The Leaky ReLU and ReLU activations converge faster and to lower final loss values than Sigmoid, demonstrating improved gradient flow and reduced saturation effects. Curves correspond to the best learning rate for each activation type.

10. Architectural Variations

To study the effect of network depth and width, we tested a series of architectures ranging from a single hidden layer to three hidden layers, with 64–256 neurons per layer. All models used ReLU activation, Adam optimizer ($\eta = 10^{-3}$), and $\lambda = 10^{-5}$.

The results (Figure 11) show a clear trend: wider networks achieve higher accuracy, while adding more hidden layers yields diminishing returns. A single hidden layer with 256 nodes achieved the best performance (98.15% test accuracy), surpassing both shallower and deeper configurations. This suggests that for moderately complex datasets like MNIST, a single sufficiently wide layer captures most of the nonlinearity required for classification.

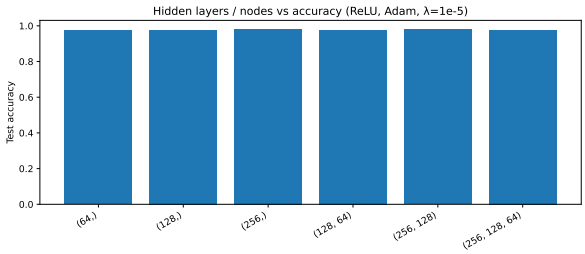

FIG. 11. Test accuracy for different network architectures (ReLU, Adam, $\lambda = 10^{-5}$). Parentheses on the x-axis indicate the number of neurons in each hidden layer. Performance improves with layer width up to 256 neurons, while adding more layers yields marginal or no gains. The single-layer (256-node) model achieves the highest accuracy, showing that width dominates depth for MNIST classification.

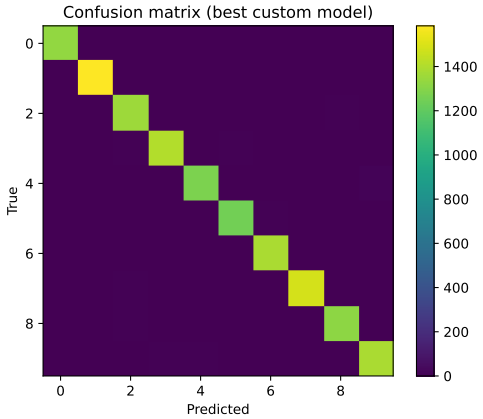

FIG. 12. Confusion matrix for the best-performing classification network (ReLU–Adam, one hidden layer with 256 neurons). Most misclassifications occur between visually similar digits such as 3/5 and 4/9, while diagonal dominance indicates strong class-level consistency.

11. *Best Model and Baseline Comparison*

The best-performing configuration was the ReLU–Adam network with one hidden layer of 256 nodes, learning rate $\eta = 10^{-3}$, and regularization $\lambda = 10^{-5}$. The detailed classification report and confusion matrix are presented below.

The model achieved an overall test accuracy of 98.15%, with precision, recall, and F1-scores all around 0.98 for most classes. Misclassifications were mainly observed between visually similar digits such as “3” and “5” or “4” and “9”. The confusion matrix (Figure 12) demonstrates balanced performance across classes and no significant bias.

Finally, our network was compared against Scikit-learn’s `MLPClassifier` baseline (ReLU, Adam, 30 epochs), which achieved 97.94% test accuracy. Both models showed nearly identical convergence trends (Figure 13), confirming the correctness and efficiency of our custom implementation.

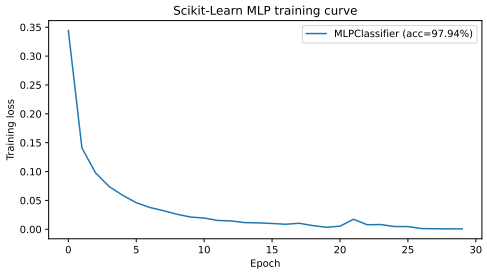

FIG. 13. Training loss curve for the Scikit-Learn `MLPClassifier` baseline (ReLU–Adam, one hidden layer with 256 neurons). The convergence behavior mirrors that of our custom implementation, confirming the numerical validity and comparable efficiency of the developed framework.

12. *Summary of Classification Findings*

- **Regularization:** Small λ (10^{-6}–10^{-7}) improves generalization; larger values underfit.
- **Learning Rate:** $\eta = 10^{-3}$ yields faster convergence and higher accuracy than 5×10^{-4}.
- **Activation Functions:** Leaky ReLU performs best, followed closely by ReLU; Sigmoid underperforms.
- **Architecture:** A single hidden layer with 256 neurons is optimal for MNIST; deeper models add minimal benefit.
- **Optimizer:** **Adam** consistently provides smoother and faster convergence than RMSProp and remains the best choice for classification.

- **Performance:** Custom NN achieves 98.15% accuracy, slightly outperforming the Scikit-Learn MLP baseline (97.94%).

Overall, the classification experiments validate both the numerical correctness and generalization capability of our neural network implementation. They further show that for balanced datasets such as MNIST, architectural width, adaptive optimization, and ReLU-family activations are the dominant factors in achieving high accuracy, while strong regularization or excessive depth offers limited additional benefit.[8]

V. CONCLUSION

This project successfully implemented a fully functional feedforward neural network (FFNN) framework from first principles, including custom activation, cost, and optimization modules. The correctness of the backpropagation and gradient computation routines was verified through comparison with automatic differentiation using `autograd`, showing negligible numerical deviation across all layers. Furthermore, all cost functions demonstrated consistent batch scaling and label-format invariance. All of the codes for the testing can be found in the Project2 folder in [5].

A. Training Configuration and Preliminary Analysis

The training stability experiments indicated that moderate batch sizes ($N_{\text{batch}} = 32$) provided the best balance between convergence speed and generalization, as seen in Figure 1, and this setting was therefore used throughout subsequent analyses. A learning rate of $\eta = 0.01$ with the Adam optimizer yielded smooth and reliable convergence across all tested configurations. This was used because we needed a reliable optimizer with a general learning rate that would expose the architecture differences, rather than us having to focus on the optimization, that we did later.

Interestingly, as seen later in the architecture heatmaps, the best-performing model (2 hidden layers, 50 nodes per layer) was not the most complex one as seen in Figure 2, 3. This suggests that moderate architectural depth provides sufficient representational capacity for the Runge function while avoiding overfitting and unstable training dynamics. This was used as the standard for the regression analysis further.

B. Optimizer Performance and Learning Rate Sensitivity

Using the best-performing architecture (2 hidden layers, 50 nodes per layer), we compared the optimization performance of Vanilla Gradient Descent, RMSprop, and Adam across logarithmically spaced learning rates from 10^{-5} to 10^{-1}. Each configuration was trained for 1500 epochs using mini-batches of 32 samples.

Adam and RMSprop both exhibited robust performance across a wide range of learning rates, achieving optimal test MSE values near $\eta = 7 \times 10^{-5}$. In contrast, Vanilla Gradient Descent required a significantly higher learning rate ($\eta \approx 2 \times 10^{-3}$) to achieve comparable accuracy and showed strong sensitivity to small deviations in step size.

The near-identical minima of Adam and RMSprop confirm that adaptive moment estimation improves convergence stability and smoothness. Adam was chosen as the default optimizer in subsequent experiments due to its slightly lower MSE and greater robustness. At higher learning rates, RMSprop exhibited unstable behavior and divergence, highlighting Adam's superior numerical stability, as supported by [9].

C. Tensorflow comparison

When comparing our custom neural network (2 hidden layers, 50 nodes per layer, Adam optimizer, $\eta = 7 \times 10^{-5}$) with an equivalent TensorFlow/Keras model, the initial results were unexpected. As shown in Figure 6, the TensorFlow network failed to converge, maintaining a high loss throughout training despite identical architecture and hyperparameters.

Upon further investigation, the discrepancy was traced back to differing weight initialization schemes. While our implementation initialized weights using a standard normal distribution, TensorFlow/Keras defaults to the *Glorot Uniform* initializer [10]. After adapting the TensorFlow model to match our initialization, the two models achieved nearly identical convergence patterns and final errors.

Interestingly, our own implementation still slightly outperformed TensorFlow's version in both MSE and R^2, suggesting minor numerical differences in optimization or gradient handling. The full reproducible implementation is available in `Project2/regression_tensorflow_vs_our-NN.py` [5].

D. Comparison with Project 1 (OLS and Ridge Regression)

To contextualize the performance of our neural network, we compared it against the best results from

Project 1, where OLS and Ridge regression models were applied to the same Runge function. While the polynomial models achieved minimum test errors around MSE ≈ 0.09, our neural network reduced this by nearly an order of magnitude, reaching MSE $= 0.0105$ and $R^2 = 0.89$. This improvement highlights the network's ability to approximate nonlinear functions without relying on fixed polynomial bases, instead learning adaptive internal representations directly from data.

Additionally, the neural network avoided the underfitting behavior observed in the polynomial models, particularly near the function extrema where OLS and Ridge struggled to capture curvature. These findings confirm that even relatively shallow neural architectures (2 layers, 50 nodes each) offer significantly greater expressive power and flexibility compared to linear regression methods used in Project 1.

E. Classification summary and analysis

Extending the same neural network framework to the multiclass MNIST classification task confirmed the generality and robustness of our implementation. The network achieved a peak test accuracy of **98.15%** using the Adam optimizer, ReLU activation, and a single hidden layer of 256 neurons, which slightly outperformed the Scikit-Learn `MLPClassifier` baseline (97.94The L2-regularization study (Figure 9) showed that small but nonzero penalty strengths ($\lambda! \approx! 10^{-5}$) improved generalization by reducing overfitting. Activation function experiments (Figure 10) demonstrated that ReLU and Leaky ReLU converged faster and achieved higher accuracy than Sigmoid, consistent with their superior gradient propagation properties. Architectural comparisons (Figure 11) revealed diminishing returns from adding more layers beyond two, with wider single-layer networks often performing best on this relatively simple visual dataset. Finally, the confusion matrix of the best model (Figure 12) indicated uniformly high precision and recall across all classes, verifying that the model generalized well without class imbalance or overfitting issues. Overall, these results validate that the same network architecture and optimization principles that performed well for regression also extend effectively to classification, underscoring the flexibility and correctness of our custom neural network implementation.

AI DISCLAIMER

This project has also made limited use of AI tools, such as ChatGPT, as a support mechanism for improving clarity, refining explanations, and troubleshooting code. The primary analysis, problem solving, and discussion were carried out independently by the project members. AI assistance was used only to enhance language in the report, verify understanding, and used for polishing figure names, ensuring that all team members fully comprehend the underlying concepts and methods.

Acknowledgments

We would like to thank our lecturer, Morten Hjorth-Jensen, for his valuable guidance and insights throughout the course. We also extend our gratitude to our group teachers for their helpful feedback and discussions during the project.

Appendix A: References

[1] Morten Hjorth-Jensen. Week 41: Neural networks and constructing a neural network code. https://compphysics.github.io/MachineLearning/doc/LectureNotes/_build/html/week41.html, 2025. Accessed: 2025-11-10.

[2] Morten Hjorth-Jensen. Week 42: Constructing a neural network code with examples. https://compphysics.github.io/MachineLearning/doc/LectureNotes/_build/html/week42.html, 2025. Accessed: 2025-11-10.

[3] Morten Hjorth-Jensen. Week 43: Deep learning: Constructing a neural network code and solving differential equations. https://compphysics.github.io/MachineLearning/doc/LectureNotes/_build/html/week43.html, 2025. Accessed: 2025-11-10.

[4] Hojjat Khodabakhsh. Mnist dataset. kaggle dataset id: hojjatk/mnist-dataset. https://www.kaggle.com/datasets/hojjatk/mnist-dataset, 2019. Accessed: 2025-11-10.

[5] Sebastian Koranda and collaborators. Project 2. https://github.uio.no/sebasmk/fys-stk4155/tree/3c72d5c70ff9028a2fa889482901b962d11ff3cf/projects/project2, 2025. GitHub repository. Retrieved November 7, 2025.

[6] scikit-learn developers. Mlpclassifier — scikit-learn 1.7.2 documentation. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html, 2025. Accessed: 2025-11-10.

[7] Yoan Tutunarov, Vladislav Foss, and Sebastian Koranda. Project 1 (fys-stk4155) — regression and optimization analysis. University of Oslo, Department of Physics, Course: FYS-STK4155, October 2025.

[8] Michael A. Nielsen. "how the backpropagation algorithm works" (chapter 2 of *neural networks and deep learning*). http://neuralnetworksanddeeplearning.com/chap2.html, 2015. Accessed: 2025-11-10.

[9] Morten Hjorth-Jensen. Optimization: The central part of any machine learning algorithm , momentum based gd. University of Oslo Lecture Notes, https://compphysics.github.io/MachineLearning/doc/LectureNotes/_build/html/chapteroptimization.html, 2025. Accessed September 2025.

[10] Keras Documentation. Dense Layer — TensorFlow/Keras API Reference. https://keras.io/api/layers/core_layers/dense/, 2025. Accessed November 2025.